Margin Notes

AI Study Assistant using RAG: embeddings, ChromaDB and a Groq LLM

Anushay Ayat | Artificial Intelligence and Data Science, SMIT

Study notes are hard to search

The problem

Five PDF chapters. Finding one answer means scrolling through all of them, and a general chatbot may answer from outside your notes.

The goal

Ask a question in plain language and get an answer grounded in the chapters, with the file and page number it came from.

The brief

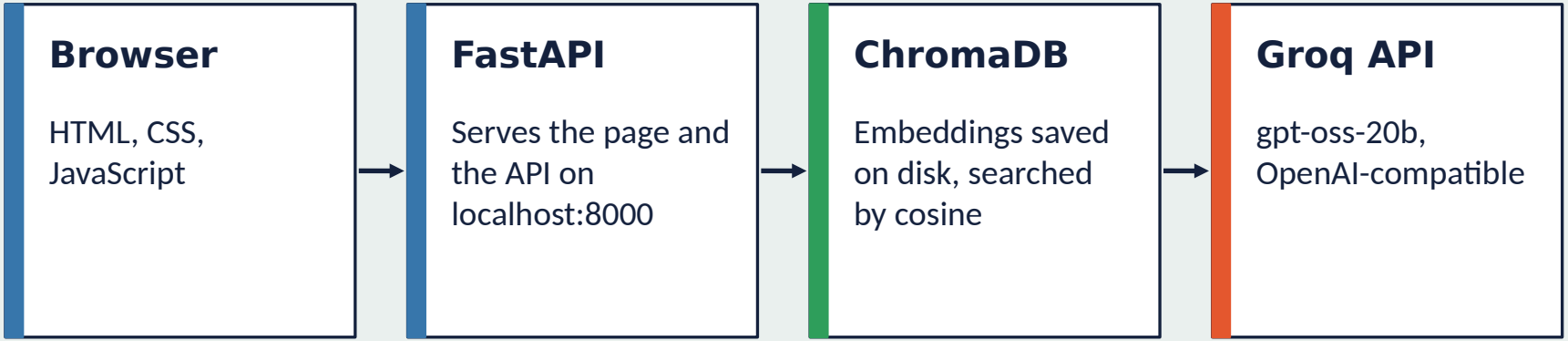
Upload 5 PDFs, split into chunks, create embeddings, save them in ChromaDB, retrieve with RAG, and answer with an LLM (Groq) through a FastAPI web app.

How RAG works in this app



Because the model only sees retrieved text, answers stay tied to your notes. If nothing relevant is found, the app says so instead of guessing.

Architecture



Firebase Auth

Email and password, or Google. The backend checks the login token on every request.

Firestore

Saves each user's chat history so it comes back after login.

Ingestion and retrieval settings

Step	Choice
Text extraction	pypdf, page by page, so every chunk keeps its page number
Chunking	900 characters with 150 overlap
Embedding model	all-MiniLM-L6-v2, 384 dimensions, runs locally
Vector store	ChromaDB, persistent on disk, cosine distance
Retrieval	Top 5 chunks, joined with recent chat for follow-ups
Answering	Groq gpt-oss-20b, temperature 0.2, answers only from the context, cites [file p.N]

What the user can do

Log in

Email and password, or Google

Ask with memory

Follow-ups like 'give an example' work

Three languages

English, Roman Urdu or Urdu answers

RAG Inspector

Chunks, embeddings, similarity scores

Quiz and study

MCQs, chapter summaries, flashcards

Speak, copy, export

Voice, copy button, PDF export, light and dark theme

Keeping answers honest

Not-in-notes guard

If the best chunk's similarity is below 0.2, the app says it could not find the answer in the notes instead of guessing. The threshold is a setting.

Retrieval evaluation

10 test questions with the PDF that should answer each.
Result on the sample chapters: 100% top-1, 100% top-5, MRR 1.0.

Automated tests

16 offline checks pass: upload, retrieval, citations, guard, follow-ups, quiz, summaries, flashcards, inspector and evaluation.

Running the app locally

1. **Set up Firebase** — Create the project, enable Auth (Email and Google) and Firestore, paste the web config
2. **Install and index** — pip install, add the Groq key to .env, index the PDFs
3. **Run** — uvicorn main:app, then open localhost:8000
4. **Test** — Log in, ask 3 questions, run the quiz and evaluation, take screenshots

Demo flow

Live demo

1. Log in with Google.
2. Upload the 5 chapters and show the embeddings being created.
3. Open the RAG Inspector for chunks and vector DB stats.
4. Ask three questions and show the cited answer.
5. Run the quiz and the evaluation.

Questions to ask

What are Python variables?
Explain the difference between a list and a tuple.
What is a for loop?
How do functions work in Python?
What are conditional statements?

Lessons and next steps

What I learned

Chunk size and overlap change answer quality. Showing the page number makes answers easy to trust. A similarity threshold stops confident wrong answers.

Next steps

OCR for scanned PDFs, separate collections per user, re-ranking of retrieved chunks, and a larger evaluation set.

GitHub: github.com/scientistayat-engineer/ai-study-assistant